

Lab 6 Excercise

Surya Prabhav Gurram, Sai Venkat Reddy Seri, Soumith Reddy Asani

November 15, 2025

Deliverable 2 - Running YOLO w/ Ultralytics

Objective

The goal of this deliverable is to demonstrate a working installation of YOLOv11 via the Ultralytics toolkit and to run four core perception tasks on a single still frame: detection, segmentation, classification, and oriented bounding box (OBB) detection. I use a single frame from the TUM RGB-D Freiburg3 Teddy dataset so the grader can compare the results directly across all tasks. I also document how to derive an OBB from a segmentation mask when the default pretrained OBB weights do not include the “teddy bear” class.

Data and Frame Extraction

The RGB video is located locally. I extracted a clean still frame at the 2-second mark using:

```
ffmpeg -i rgbd_dataset_freiburg3_teddy_camera_rgb_image_color.mp4 \
  -ss 00:00:02 -vframes 1 teddy_frame.jpg
```

The resulting file `teddy_frame.jpg` is used as the input to all YOLO tasks.

Concepts

Detection finds objects and gives bounding boxes with confidence scores. It is efficient because YOLO predicts locations and class probabilities in one forward pass.

Segmentation assigns a label to each pixel. YOLO segmentation models output instance masks that provide fine-grained object contours.

Classification predicts only the image-level label without localization. Useful when positional information is not needed.

Oriented bounding box (OBB) predicts rotated boxes aligned to the object’s pose. When pretrained OBB models lack the desired class, we can derive an OBB by fitting a rotated rectangle (`cv2.minAreaRect`) to a segmentation mask.

Environment and Installation Notes

Ultralytics was installed using system Python:

```
python3 -m pip install --upgrade pip --break-system-packages
python3 -m pip install ultralytics --break-system-packages
```

If the `yolo` command is not found:

```
echo 'export PATH="$PATH:$HOME/.local/bin"' >> ~/.bashrc
source ~/.bashrc
```

Commands Used for Each Task

Detection

```
yolo predict task=detect model=yolo11n.pt \
  source=teddy_frame.jpg device=cpu save=True
```

Segmentation

```
yolo predict task=segment model=yolo11m-seg.pt \
  source=teddy_frame.jpg device=cpu conf=0.12 save=True save_txt=True
```

Classification

```
yolo predict task=classify model=yolo11n-cls.pt \
  source=teddy_frame.jpg device=cpu save=True
```

Oriented Bounding Box (OBB): two approaches

Direct OBB model:

```
yolo predict task=obb model=yolo11m-obb.pt \
  source=teddy_frame.jpg device=cpu conf=0.15 save=True
```

This typically gives no teddy detections since OBB weights don't include indoor classes.

Derived OBB from segmentation mask:

Using OpenCV:

```
cv2.minAreaRect(contour_points)
```

Generates a rotated box, center point, and rotation angle.

Representative Results

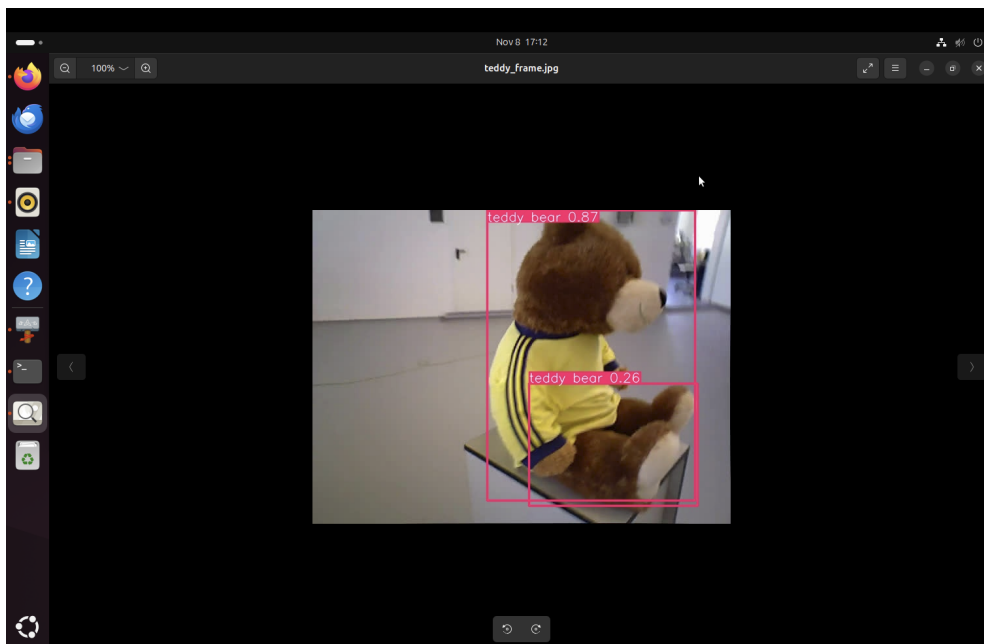


Figure 1: YOLO detection result on the teddy frame.

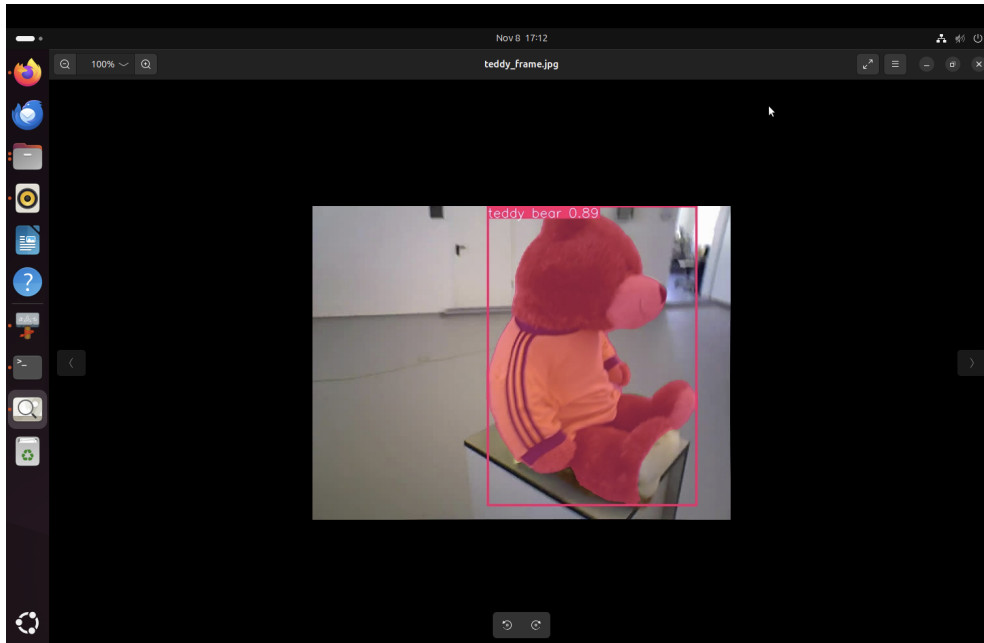


Figure 2: Segmentation output with instance mask.

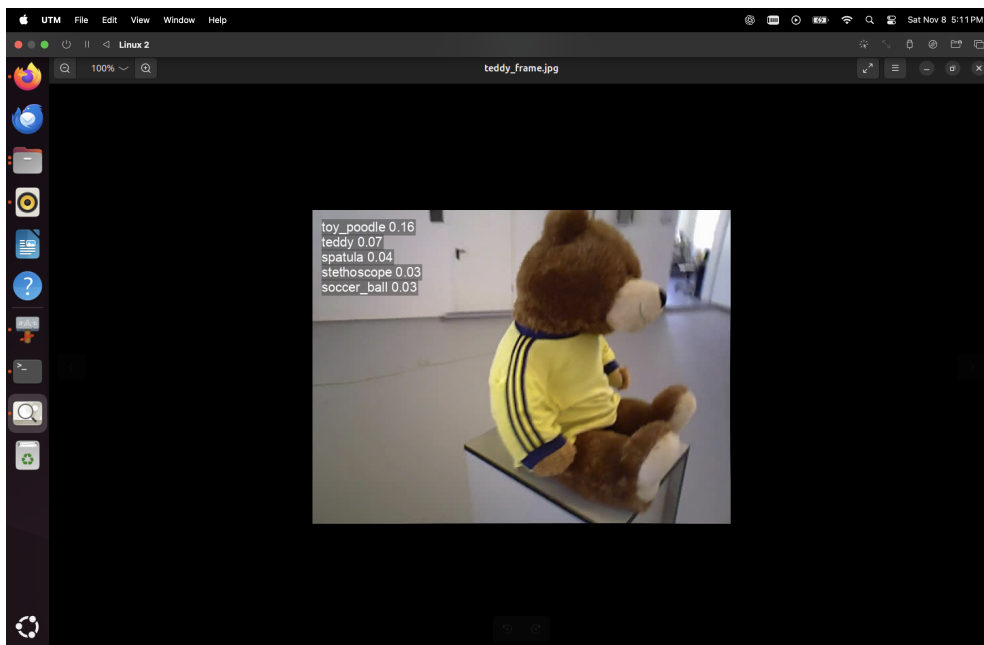


Figure 3: Classification result showing top image-level label.

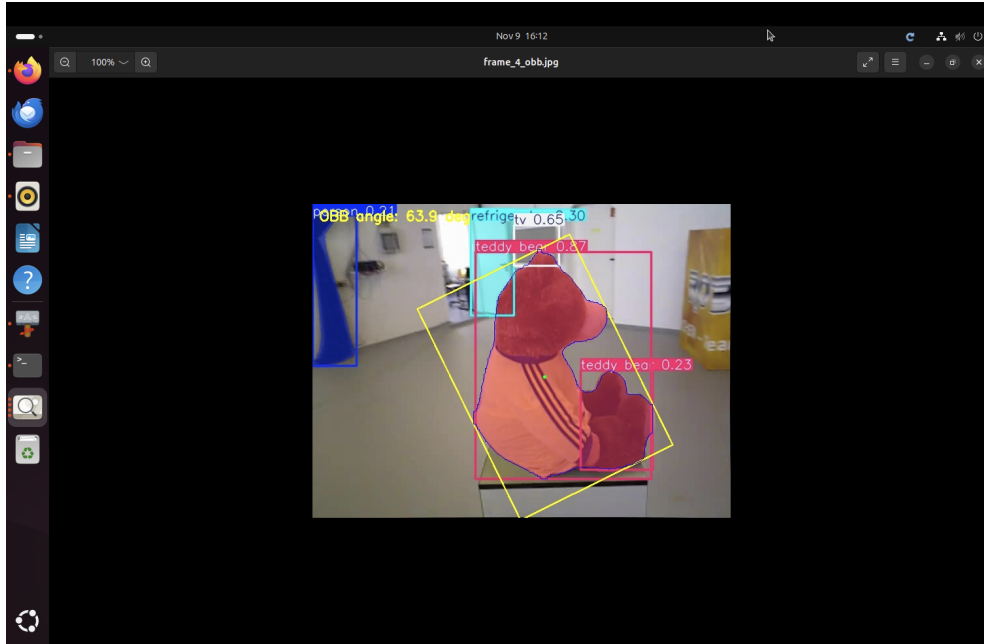


Figure 4: Oriented bounding box derived from segmentation mask.

Deliverable 3 - Object Localization

Objective

The goal of this deliverable was to estimate the 3D position of the teddy bear in the `rgbd_dataset_freiburg3_teddy` sequence by combining YOLO detections with the camera poses (from TF) and the camera intrinsics. Using these measurements, we built a small GTSAM factor graph to triangulate a single 3D teddy landmark and visualized the result in RViz. The optional part includes computing and visualizing the covariance of the teddy estimate.

Implementation

Our node performs three main steps:

- **1. Running the bag + YOLO:** The dataset is played with:

```
ros2 bag play -l ~/vnav/data/rgbd_dataset_freiburg3_teddy
```

YOLOv11 (from `ultralytics_ros`) publishes detections on `/yolo_result` and annotated images on `/yolo_image`.

- **2. Collecting measurements:** We subscribe to:

- `/camera/rgb/camera_info` → camera intrinsics
- TF → transform from `world` to `openni_rgb_frame`
- `/yolo_result` → bounding box center (u, v)

For every detection labeled “teddy” or “bear,” we store:

$$(u, v, T_{cam}^{world})$$

as a single 2D–3D measurement.

- **3. GTSAM factor graph:** We introduce:
 - A single 3D landmark L (the teddy)
 - A camera pose variable x_i for each valid detection

For each pose we add:

- A **PriorFactor** using the TF camera pose
- A **GenericProjectionFactor** connecting x_i , L , and the pixel (u_i, v_i)

The graph is optimized using Levenberg–Marquardt to estimate the teddy’s 3D location.

After optimization the node publishes:

- `/bear_pose` – estimated teddy landmark
- `/bear_pose_cov` – optional covariance ellipsoid
- `/cortex_marker_array` – green camera path, red arrows, purple sphere

Results

The GTSAM optimization produced a stable 3D teddy estimate. As the camera moved, the estimated teddy landmark remained fixed in the `world` frame, and RViz showed:

- Green line → full camera trajectory
- Red arrows → poses that contributed YOLO detections
- Purple sphere → final teddy 3D position
- Lower-left YOLO panel → live teddy bounding box

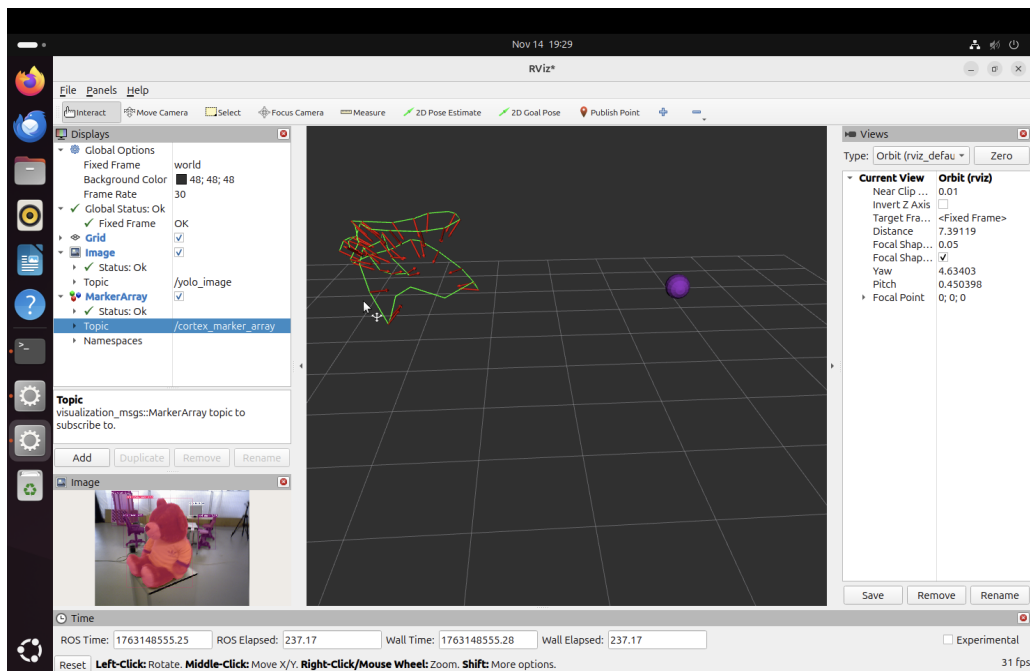


Figure 5: Deliverable 3 – RViz visualization of camera trajectory (green), pose arrows (red), and teddy landmark (purple).

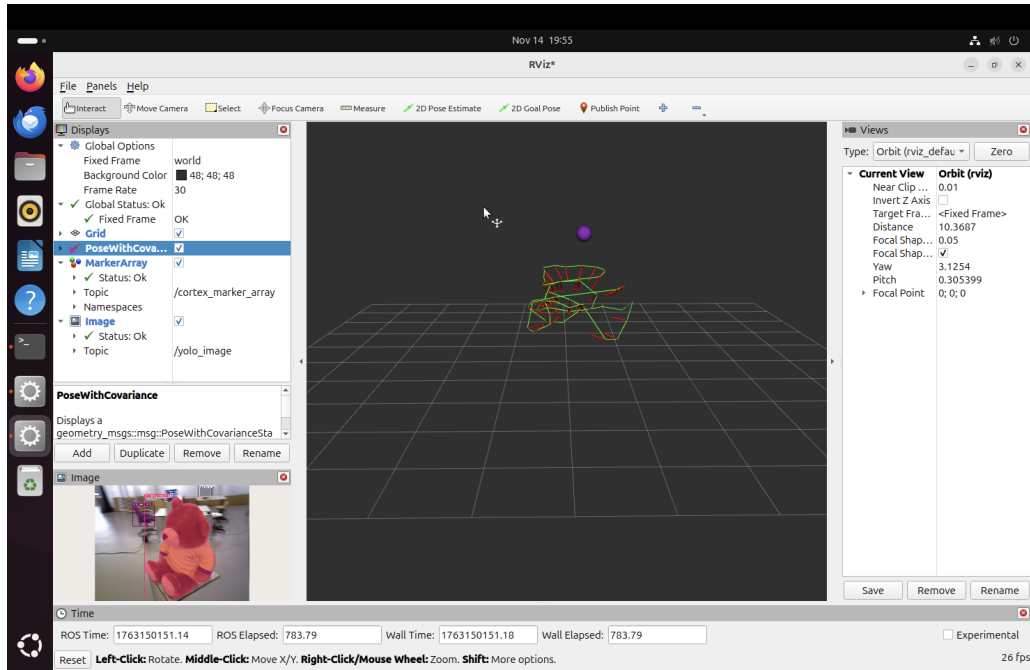


Figure 6: Second RViz view during teddy localization. YOLO detection is displayed on the image panel.

Best Teddy Bear Estimate

The final optimized 3D position of the teddy bear is obtained from the `/bear_pose` topic (a `geometry_msgs::PoseStamped` message). The position component corresponds to the required `geometry_msgs::PointStamped` “best estimate”:

Best Teddy Bear 3D Estimate (`PointStamped`, `frame_id = "world"`):

```
x: 0.8999
y: -0.3155
z: 1.5431
```

Optional: Covariance Visualization

For the optional task, we compute the covariance:

```
gtsam::Marginals marg(graph, result);
gtsam::Matrix3 cov = marg.marginalCovariance(L);
```

This covariance is placed into a `PoseWithCovarianceStamped` message and published on `/bear_pose_cov`. RViz displays it as an ellipsoid showing how confident the estimate is.

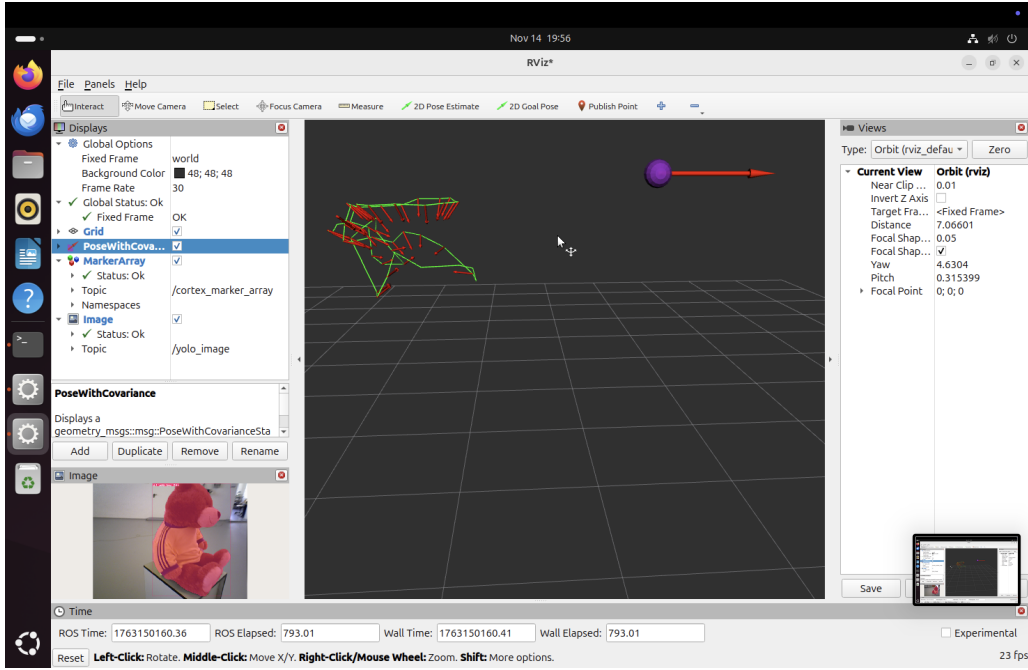


Figure 7: Optional covariance visualization using PoseWithCovariance. The ellipsoid represents uncertainty in the teddy's 3D location.

Covariance Matrix of Teddy Bear Estimate

From the `/bear_pose_cov` topic (a `geometry_msgs::PoseWithCovarianceStamped` message), we extract the top-left 3×3 translational block of the 6×6 covariance matrix. A representative covariance for the teddy bear landmark is:

Covariance (3x3) in the world frame:

$$\begin{bmatrix} 2.0592\text{e-}08 & 2.4869\text{e-}09 & 1.2145\text{e-}09 \\ 2.4869\text{e-}09 & 3.5592\text{e-}09 & 7.7876\text{e-}10 \\ 1.2145\text{e-}09 & 7.7876\text{e-}10 & 2.3331\text{e-}10 \end{bmatrix}$$

These very small values indicate that the 3D teddy position is well constrained by the available measurements.